

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

Н. В. Богословская
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 3

ПОДГОТОВКА ПРИЗНАКОВ

по курсу:

БОЛЬШИЕ ДАННЫЕ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4326

подпись, дата

Г. С. Томчук
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: изучить методы подготовки признаков для анализа больших данных и освоить их реализацию в платформе Loginom с использованием визуальных компонентов и узла Python.

2 Задание

В ходе выполнения работы необходимо выполнить следующие задачи:

1. Удаление и заполнение пропущенных значений
2. Обнаружение и обработка выбросов
3. Масштабирование признаков (нормализация и стандартизация)
4. Отбор признаков на основе корреляционного анализа

Методы подготовки признаков реализовать с использованием Python и встроенных инструментов Loginom. Сравнить подходы обработки данных и сформулировать выводы о преимуществах различных методов.

3 Ход выполнения

В качестве исходных данных для выполнения лабораторной работы был использован набор данных `framingham.csv`, содержащий медицинские показатели пациентов и целевой признак `TenYearCHD`, отражающий вероятность развития ишемической болезни сердца в течение 10 лет.

На первом этапе была выполнена загрузка и предварительный анализ набора данных. Аналитический сценарий обработки данных в платформе Loginom представлен на рисунке 1.

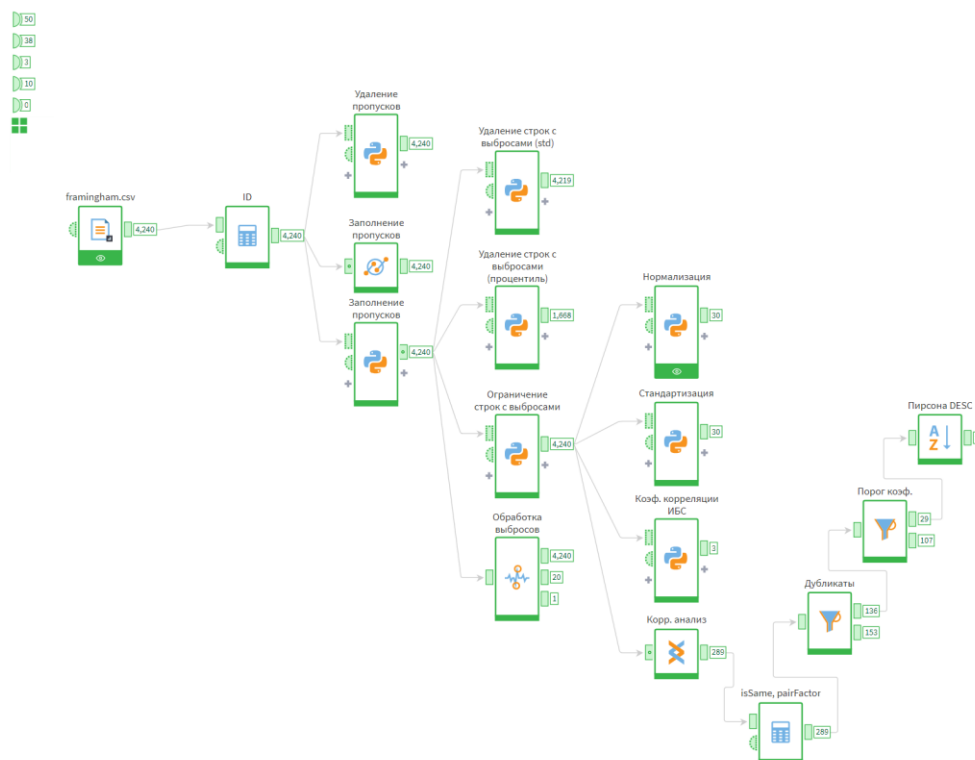


Рисунок 1 — Аналитический сценарий

Далее была проведена обработка пропущенных значений двумя способами: с использованием визуального узла платформы и средствами Python. Для дискретных признаков заполнение пропусков осуществлялось модой, так как такие признаки представлены категориальными значениями. Для числовых признаков использовалось заполнение медианой, поскольку данный метод менее чувствителен к выбросам по сравнению со средним значением. Пример обработки пропусков представлен на рисунке 2.

```

1 import builtin_data
2 from builtin_data import InputTable, InputTables, InputVariables, OutputTable, OutputTables, DataType, DataKind, UsageType
3
4 import numpy as np, pandas as pd
5 from builtin_pandas_utils import to_data_frame, prepare_compatible_table, fill_table
6
7 if InputTable:
8     # Создать pd.DataFrame по входному набору
9     data = to_data_frame(InputTable)
10
11 #data["education"] = data["education"].fillna(data["education"].max()).astype(int)
12
13 #data["BPMeds"] = data["BPMeds"].fillna(data["BPMeds"].min()).astype(int)
14 #data["totChol"] = data["totChol"].fillna(data["totChol"].min()).astype(int)
15
16 #data["BMI"] = data["BMI"].fillna(20).astype(int)
17
18 #data["heartRate"].fillna(data["heartRate"].median(), inplace=True)
19 #data["glucose"].fillna(data["glucose"].median(), inplace=True)
20
21 # Заполнение дискретных/бинарных признаков модой (самым частым значением)
22 features = ["male", "education", "currentSmoker", "BPMeds", "prevalentStroke", "prevalentHyp", "diabetes"]
23 for feature in features:
24     data[feature] = data[feature].fillna(data[feature].mode()[0]).astype(int)
25
26 data.fillna(data.median(numeric_only=True), inplace=True)
27
28 # Полученный выходной pd.DataFrame
29 output_frame = pd.DataFrame(data)
30
31 # Если включена опция "Разрешить формировать выходные столбцы из кода", структуру выходного набора можно подготовить по pd.DataFrame
32 if isinstance(OutputTable, builtin_data.ConfigurableOutputTableClass):
33     prepare_compatible_table(OutputTable, output_frame, with_index=False)
34 fill_table(OutputTable, output_frame, with_index=False)
35

```

Рисунок 2 — Заполнение пропусков модой и медианой

После обработки пропусков была выполнена работа с выбросами. С помощью Python были реализованы методы удаления строк с выбросами на основе стандартного отклонения, удаления выбросов с использованием процентилей, а также ограничение экстремальных значений без удаления строк. Дополнительно была проведена обработка выбросов средствами узла «Редактирование выбросов» платформы Loginom. Пример обработки выбросов представлен на рисунке 3.

```

1 import builtin_data
2 from builtin_data import InputTable, InputTables, InputVariables, OutputTable, OutputTables, DataType, DataKind, UsageType
3
4 import numpy as np, pandas as pd
5 from builtin_pandas_utils import to_data_frame, prepare_compatible_table, fill_table
6
7 if InputTable:
8     # Создать pd.DataFrame по входному набору
9     data = to_data_frame(InputTable)
10
11 upper_limit = data["cigsPerDay"].quantile(0.95)
12 lower_limit = data["cigsPerDay"].quantile(0.05)
13 data.loc[(data["cigsPerDay"] > upper_limit), "cigsPerDay"] = upper_limit
14 data.loc[(data["cigsPerDay"] < lower_limit), "cigsPerDay"] = lower_limit
15
16 # Полученный выходной pd.DataFrame
17 output_frame = pd.DataFrame(data)
18
19 # Если включена опция "Разрешить формировать выходные столбцы из кода", структуру выходного набора можно подготовить по pd.DataFrame
20 if isinstance(OutputTable, builtin_data.ConfigurableOutputTableClass):
21     prepare_compatible_table(OutputTable, output_frame, with_index=False)
22 fill_table(OutputTable, output_frame, with_index=False)
23

```

Рисунок 3 — Ограничение выбросов вне диапазона процентилей 5–95

На следующем этапе было выполнено масштабирование признаков

средствами Python. Для анализа были реализованы методы нормализации и стандартизации данных. Нормализация использовалась для приведения значений к диапазону от 0 до 1, а стандартизация — для преобразования распределения признаков к виду со средним значением 0 и стандартным отклонением 1. Результаты нормализации представлены на рисунке 4, результаты стандартизации — на рисунке 5.

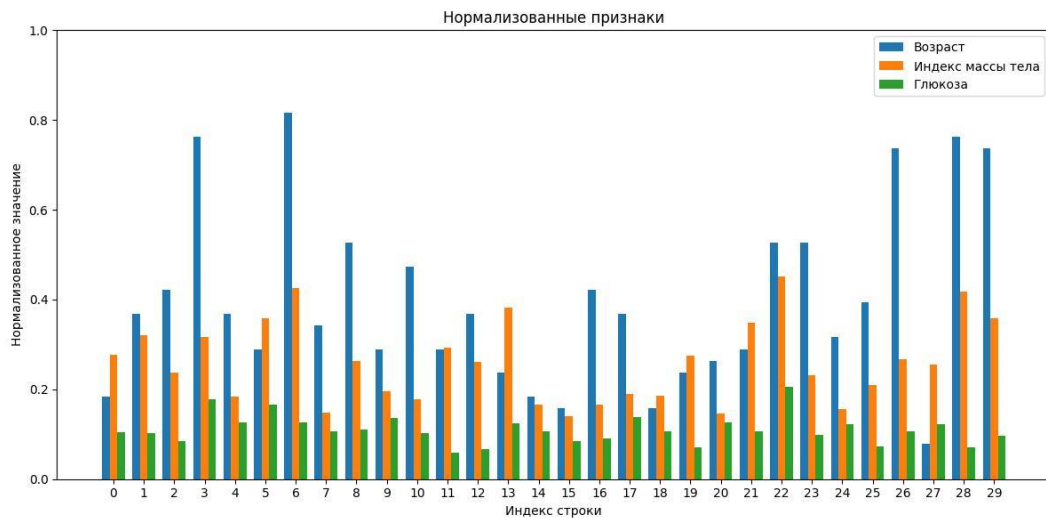


Рисунок 4 — Результаты нормализации

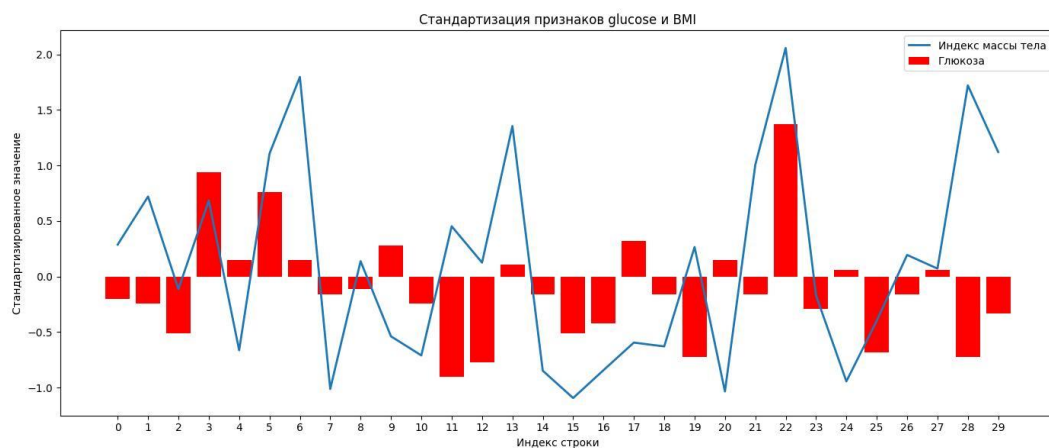


Рисунок 5 — Результаты стандартизации

Далее был проведен корреляционный анализ признаков. С помощью Python были рассчитаны коэффициенты корреляции Пирсона между признаками и целевой переменной TenYearCHD. В результате анализа была выявлена наиболее заметная связь риска сердечно-сосудистого заболевания с признаками age, sysBP и prevalentHyp. Результаты корреляционного анализа представлены на рисунке 6.

Козф. корреляции ИБС • Быстрый п		
Выходной набор данных		
#	ab Признак	9.0 Корреляция
1	age	0.23
2	sysBP	0.22
3	prevalentHyp	0.18

Рисунок 6 — Результаты корреляционного анализа (риск ИБС)

Дополнительно корреляционный анализ был выполнен средствами узла «Корреляционный анализ» платформы Loginom. Были рассчитаны коэффициенты корреляции между всеми признаками набора данных. После расчета были удалены дублирующие пары признаков и отфильтрованы значения коэффициентов корреляции ниже порога 0.15. В результате были выявлены значимые зависимости между признаками currentSmoker и cigsPerDay, sysBP и diaBP, а также sysBP и prevalentHyp, и прочими. Результаты анализа представлены на рисунке 7.

Выходной набор данных							
#	0/1 isSame	ab Поле1.Имя	ab Поле1.Метка	ab Поле2.Имя	ab Поле2.Метка	9.0 Пирсона	0/1 pairFactor
1	false	currentSmoker	currentSmoker	cigsPerDay	cigsPerDay	0.81	false
2	false	sysBP	sysBP	diaBP	diaBP	0.78	false
3	false	sysBP	sysBP	prevalentHyp	prevalentHyp	0.70	false
4	false	prevalentHyp	prevalentHyp	diaBP	diaBP	0.62	false
5	false	glucose	glucose	diabetes	diabetes	0.61	false
6	false	sysBP	sysBP	age	age	0.39	false
7	false	diaBP	diaBP	BMI	BMI	0.38	false

Рисунок 7 — Фрагмент результирующей таблицы корреляционного анализа

4 Выводы

В ходе выполнения лабораторной работы были рассмотрены методы подготовки признаков, применяемые при анализе данных. Были выполнены операции обработки пропущенных значений, обнаружения и обработки выбросов, масштабирования числовых признаков, а также корреляционного анализа для оценки взаимосвязи переменных.

Практическая часть показала, что платформа Loginom предоставляет удобные визуальные инструменты для выполнения основных операций

предобработки данных. Использование встроенных компонентов позволяет быстро реализовывать типовые этапы подготовки данных без необходимости написания кода, что упрощает разработку и ускоряет создание аналитических сценариев.

Одновременно применение узла Python расширяет возможности обработки данных. Программный подход позволяет более гибко управлять логикой преобразования данных, реализовывать нестандартные методы обработки и комбинировать различные алгоритмы анализа.

Таким образом, использование визуальных инструментов платформы для базовых операций и программирования на Python для более сложных задач позволяет эффективно выполнять подготовку признаков и повышает гибкость процесса анализа данных.